ПОДХОДЫ К ОРГАНИЗАЦИИ БЛОКИРОВКИ МНОЖЕСТВЕННОГО ЗАПУСКА ПЕРЕОБУЧЕНИЯ МОДЕЛИ АНТИ-ФРОД ПРИ ОБНАРУЖЕНИИ СДВИГА ДАННЫХ

ПРЕЗЕНТАЦИЯ ИТОГОВОГО ПРОЕКТА OTUS / *АКИМОВ ПЕТР ИГОРЕВИЧ*

БЛОКИРОВКА REDIS VS. KAFKA

СТУДЕНТ: АКИМОВ ПЕТР ИГОРЕВИЧ

КУРС: MLOPS

ДАТА: 30.04.2026

О ПРОЕКТЕ

- Цель проекта: продемонстрировать разные подходы к организации блокировки множественного запуска переобучения модели анти-фрод при обнаружении сдвига данных
- Целевая аудитория: MLOps-инженеры
- Особенности проекта:

  - выполнено на публичной облачной платформе Яндекс Облако

  - алгоритм машинного обучения предложен автоматически с помощью AutoML инструментов (FLAML)

  - рассмотрены сценарии блокировки: через Redis, через Apache Kafka

  - реализован инференс на потоке с настройкой сдвига данных и TPS

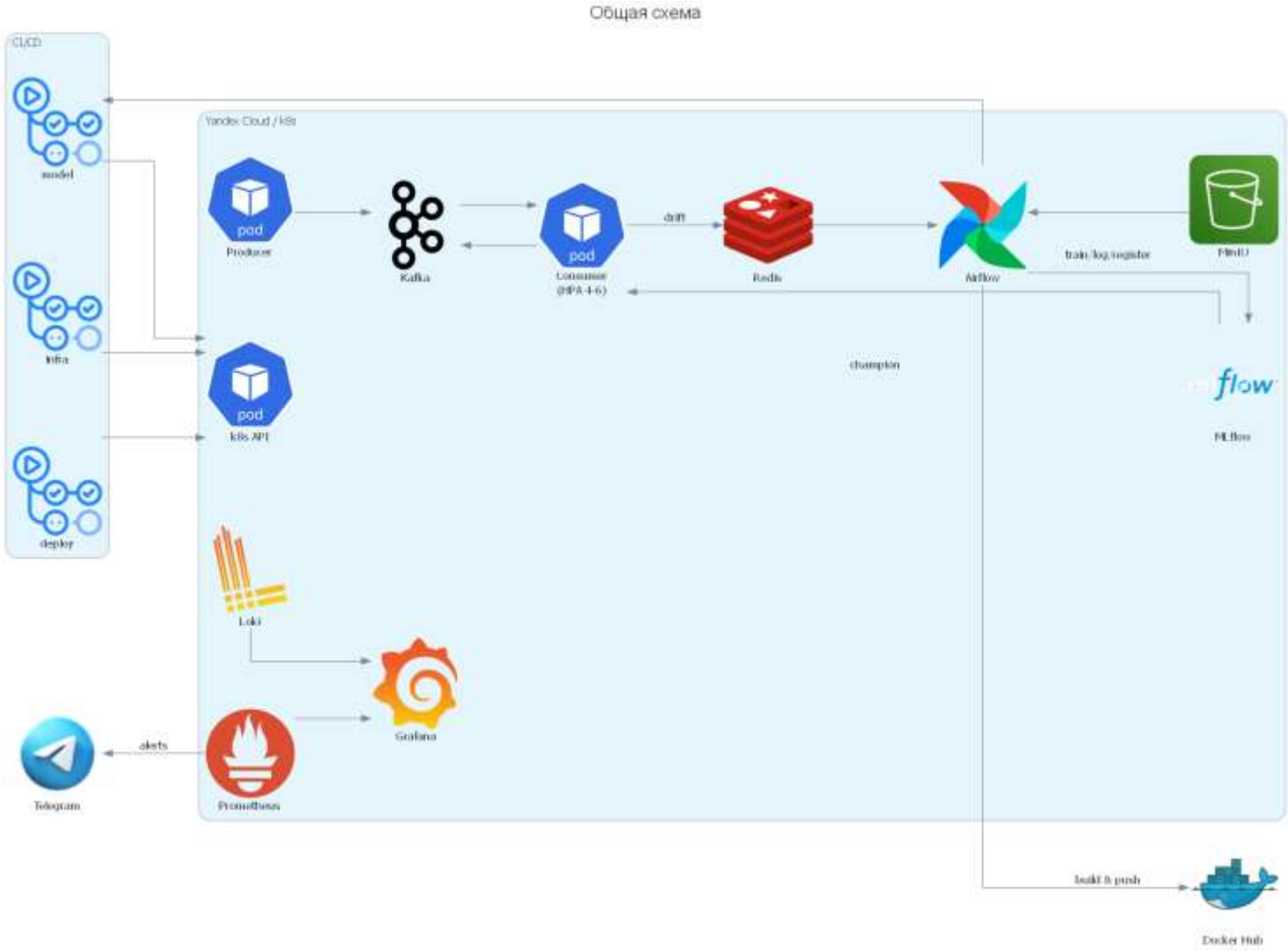
ТЕХНОЛОГИИ И ИНСТРУМЕНТЫ

1. Python
2. FLAML
3. XGBoost
4. Apache Kafka
5. MLflow
6. Apache Airflow
7. GitHub Actions
8. Kubernetes
9. Evidently
10. Redis

ЭТАПЫ РАЗРАБОТКИ

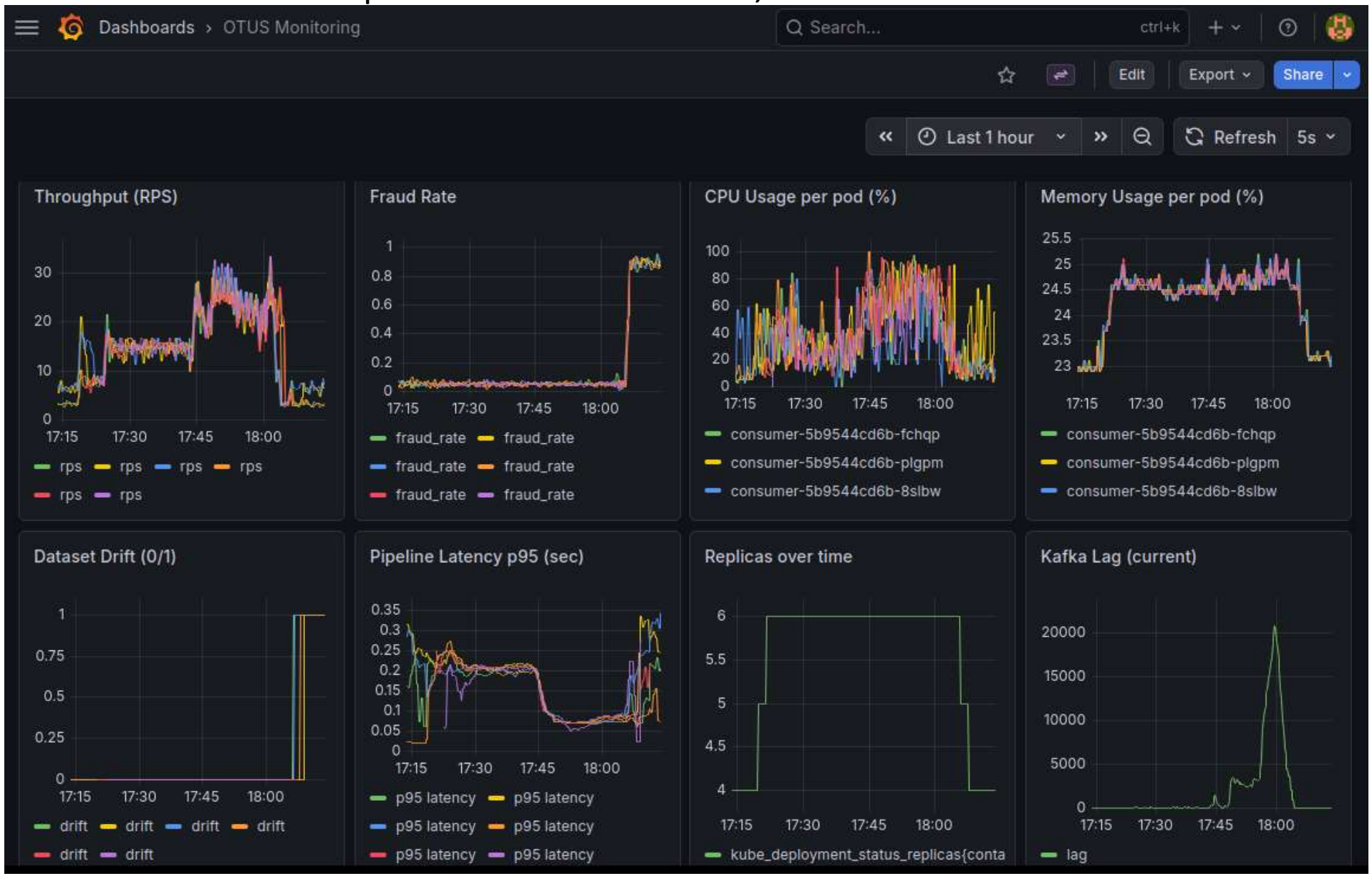
1. Подготовка данных
2. Подбор модели
3. Внедрение
4. Анализ результатов

ДЕМОНСТРАЦИЯ ПРОЕКТА, ЧАСТЬ 1. ОБЩАЯ СХЕМА



Ссылка на проект https://github.com/petr-akimov/otus-final

ДЕМОНСТРАЦИЯ ПРОЕКТА, ЧАСТЬ 2. GRAFANA



ТРУДНОСТИ И РЕШЕНИЯ

1. Реплик пода с моделью много, а POST-запрос к API CI-системы не должен дублироваться

   - Решение: использовать два подхода для блокировки POST-запросов: Redis и Kafka

2. Как эмулировать сдвиг данных в рамках учебного проекта?

   - Решение: подготовить два датасета таким образом, чтобы Evidently обнаружил сдвиг

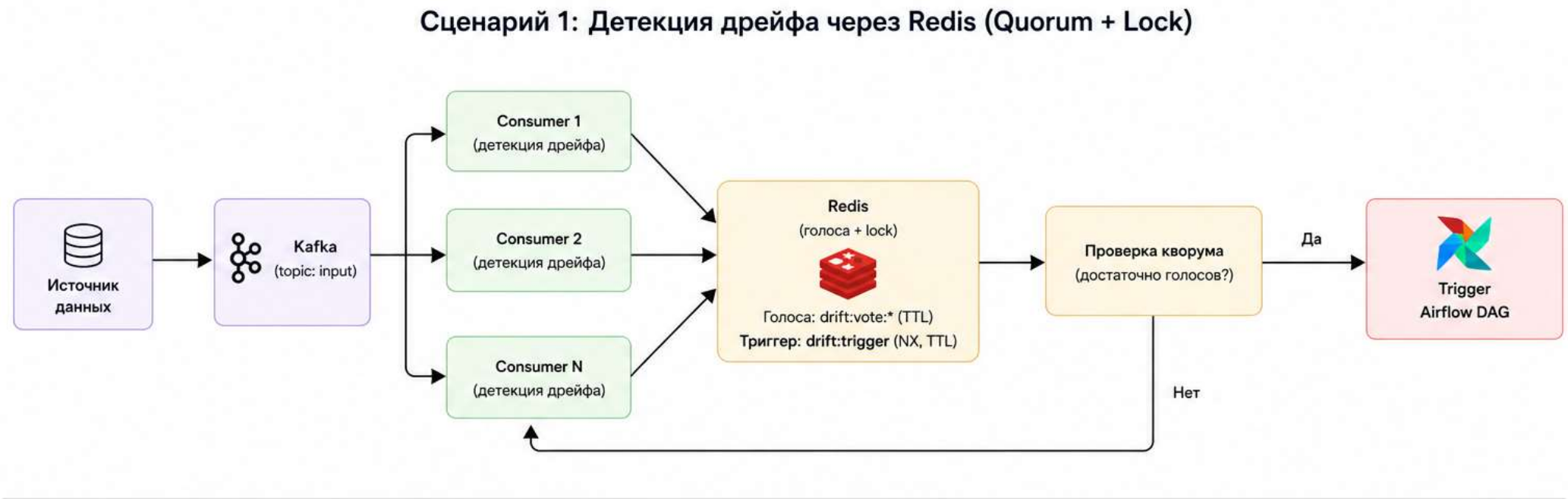
3. Алгоритмов машинного обучения много, времени мало, и присутствует человеческий фактор

   - Решение: делегировать право выбора алгоритма машинного обучения самим машинам (winner: XGBoost)

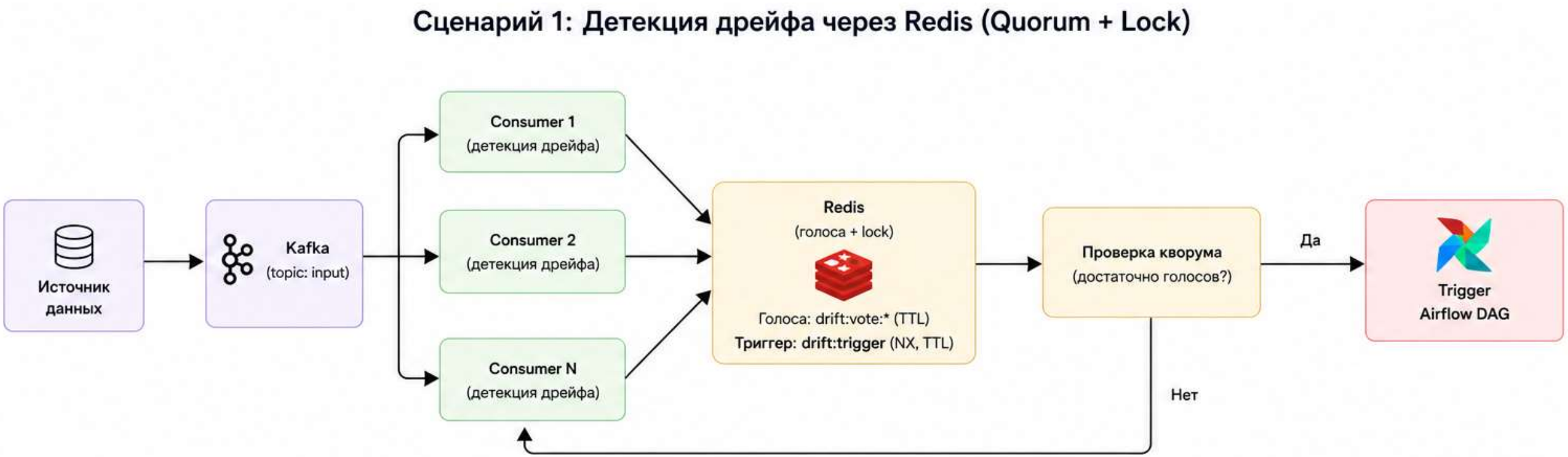
4. Если многие алгоритмы дают приблизительно один и тот же ROC-AUC > 99%, то какую метрику сделать решающей?

   - Решение: комплексная метрика (ROC-AUC > 99% + min_inference_time)

СЦЕНАРИИ 1(REDIS) И 2(KAFKA)



СЦЕНАРИЙ 1 - REDIS



СЦЕНАРИЙ 2 - KAFKA

СРАВНИТЕЛЬНАЯ ТАБЛИЦА

Критерий	Через Redis	Через Kafka
Как координировать	Все реплики «голосуют» в Redis, и только одна, первой насчитав кворум, запускает процесс через специальный флаг	Выбирается лидер, и только он решает, запускать процесс или нет
Что нужно дополнительно	Нужен отдельный Redis	Нужна только Kafka (но с дополнительными топиками)
Сложность	Просто реализовать	Сложнее: нужно управлять лидером и его сменой
Как удалять старые голоса	Автоматически (Redis сам удаляет по времени)	Лидер сам удаляет старые голоса по времени
Как избежать повторных запусков	Есть общий флаг с таймером (TTL)	Лидер следит за временем последнего запуска (cooldown)
Масштабируемость	Может тормозить при большом числе инстансов	Лучше масштабируется

ВЫВОДЫ

Оба подхода успешно предотвращают множественные запуски Airflow DAG при обнаружении дрейфа данных несколькими репликами приложения consumer. Реализация на Redis проще, использует атомарную операцию setnx (установить ключ, только если он не установлен на данный момент Set if Not eXists) для глобальной блокировки и TTL для автоматического сброса состояния, но зависит от внешнего Redis и операции keys (решение опирается на механизм keys, который плохо масштабируется и может стать bottleneck при росте нагрузки). Реализация на Kafka полностью автономна (не требует Redis) и строит распределённую координацию через выделенного лидера и дополнительные топики, однако сложнее в реализации и поддержке. Выбор конкретного решения определяется наличием Redis в инфраструктуре и требованиями к сложности процессов и стека.

БЛАГОДАРНОСТИ

- Руководителю курса и преподавателю, проверявшему домашние задания
- Авторам курса, лекторам на вебинарах
- Всему персоналу OTUS, причастному к курсу, его организации, поддержке

ВАШИ ВОПРОСЫ?

СПАСИБО

Петр Акимов

@petrakimov